Group Project - Pac-Man (Text-Based, Standard C Only)

Project report
Contribution to the overall marks: 35 %

Group ID	Member	Student ID	Individual part (own chapter)
Group6	Boyang.Bian	2577254	Part 1 - Map & Rendering
Group6	Zehao.Xin	2580224	Part 2 - Player Control & Rules
Group6	Siqi.Fan	2581778	Part 3 - Ghost Logic
Group6	Grigorii Vorobev	2582036	Part 4 - Game State & Testing

Build command (as suggested by the brief): gcc *.c -o pacman

Verified in this build: checks run: 294, failed: 0, integration tests run: 19, failed: 0.

Contents

	Chapter	Content
1	Outline (shared chapter)	assignment checklist, layout, build, architecture, data structures, state machine, tick order, reproducibility, group result overview
2	Part 1 - Map & Rendering (Boyang.Bian)	board representation, maps, loading, integrity checks, rendering, individual tests
3	Part 2 - Player Control & Rules (Zehao.Xin)	input handling, movement, wall collision, death and respawn, individual tests
4	Part 3 - Ghost Logic (Siqi.Fan)	random walk, no-reversal rule, scared ghosts, smart mode, individual tests
5	Part 4 - Game State & Testing (Grigorii Vorobev)	state machine, scoring, CLI, deterministic RNG, test strategy and results
6	Integration verification and conclusion	integration, contribution table, extensions, limitations, reflection
A	Appendix A - recorded console logs	win, losing a life, power pellet, rendered board
B	Appendix B - source inventory and test cases	files, owners, line counts, test case list

1 Outline (shared chapter)

This chapter is the shared part of the report: it explains what the assignment asks for, how the program is built and how the four individual parts fit together. Chapters 2 to 5 are the four individual parts, one chapter per member, so that every member's own work and own test results can be checked on their own.

1.1 Assignment requirements and where they are delivered

Requirement in the brief	How it is met	Evidence
Maze as a 2D char array, up to 25 x 40, symbols # . O P G and space	map.c stores the board in char board[25] [40] inside struct Game; five embedded maps (24x11 and 24x11 rotated) plus an optional text-file loader	unit tests Part 1; tests/cases/*.txt
Render the board to the console each tick, top to bottom	render.c prints one row per line followed by a status line	log board_render_demo (Appendix A.4)
Tick based loop, W/A/S/D to move, Q to quit	input.c reads one command per tick; main.c runs the loop	unit tests Part 2; integration case invalid_keys_ignored
A move into a wall keeps Pac-Man in place	entity.c entity_step() returns 0 and the position is unchanged; the tick still advances	unit test 'wall bump still advances the tick'; integration case wall_bump_no_move
After Pac-Man moves: eat pellet, then every ghost moves once	game.c game_tick() steps 1-7 (Figure 2)	unit tests Part 4; tick log in Appendix A.4
Print updated board and status (score, lives, pellets, tick)	render.c render_status() prints all four values	any recorded log

1-4 ghosts, each tick a uniformly random non-wall direction	ghost.c collects the legal directions and picks one with rng_range(); the no-reversal rule of thumb is applied unless it is the only option; a ghost that is followed for 3 ticks in a row (same corridor, up to 5 cells behind) turns round (section 4.5)	unit tests Part 3 (10000 moves never enter a wall)
Pellet +10 points and removed from the map	game.c game_consume()	unit test 'eating pellets'
Ghost collision (either direction): lose a life, respawn, pellets stay eaten	game.c collision_check() at step 3 and step 7; lose_life() + game_respawn()	integration cases die_ghost_touches_pac and die_pac_touches_ghost
3 lives, game ends at 0	game.c lose_life()	integration case die_ghost_touches_pac
Win when all pellets are eaten	game.c step 4 win check	integration case win
Randomness reproducible with a user-supplied seed	rng.c xorshift32 seeded by --seed; the C library rand() is never used	integration cases same_seed_identical and different_seed_differs
Input validation, no crash on EOF	input.c returns INPUT_INVALID / INPUT_EOF; main.c handles both	integration cases invalid_keys_ignored and eof_no_crash
Optional extensions for higher marks	power pellets, scared ghosts, chase/scatter, fruit, second level, map loader	section 6.3

1.2 Deliverables and layout of the submission

Item	Path	Notes
Report (this document)	report/report.pdf	required by the brief
Source code	*.c, *.h in the project root	gcc *.c -o pacman works directly
Build / run guide	README.md	build, options, rules, tests
Unit tests	tests/unit_tests.c	294 checks, no Python needed
Integration tests	tests/run_tests.py	14 scripted games + reference logs
Test runners	tests/run_tests.bat, tests/run_tests.sh, Makefile	one command each
Recorded logs	tests/logs/, tests/expected/	evidence used in this report
Submission archive	Group6 PacmanC.zip	brief: 'GroupID PacmanC.zip'

1.3 Build and run

The brief asks for a simple build, so the game is built exactly the way it suggests - from the project root, with every source file in that folder:

```
gcc *.c -o pacman          # the command from the brief
pacman --seed 42 --ghosts 2 # play (Windows: pacman.exe)
build.bat                   # same build on Windows, one command
tests\run_tests.bat         # build + unit tests + integration tests
```

The rebuild was repeated while writing this report. The compiler (gcc 8.1.0, MinGW-w64) produced the binary with -Wall -Wextra -pedantic and no warning:

```
$ gcc -std=c99 -Wall -Wextra -pedantic -O2 *.c -o pacman.exe
(no output: no errors, no warnings)
```

The program starts with a one line menu that asks for the difficulty. Everything that is not a digit 1-5 and not q is ignored and the question is repeated, exactly like an invalid key inside the game; end of input at the menu leaves the program cleanly instead of crashing.

```
Choose difficulty 1-5 (q to quit):
```

difficulty	levels to clear	ghosts
1 easy	1	2
2 normal	2	2
3 hard	3	3
4 harder	4	3
5 expert	5	4

The menu is a convenience only: --difficulty N sets the same values without asking, and an explicit --levels or --ghosts always wins over the table. The automated tests therefore never have to answer the menu, and one integration case exists only to test it.

1.4 Architecture and module interfaces

Figure 1 - module architecture (owner part in brackets)

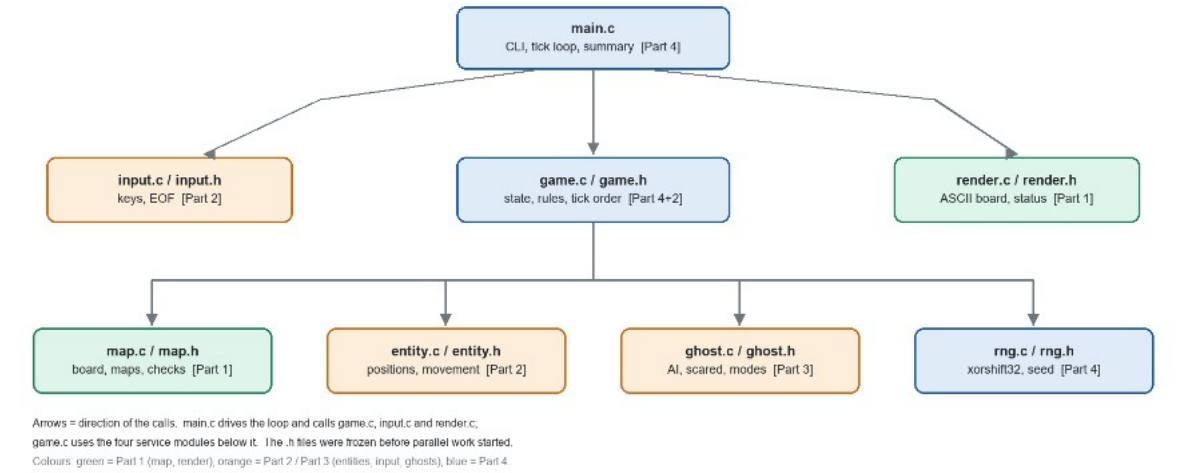


Figure 1 - eight source modules, one owner each; the arrows show the call direction.

Module	Public interface (the .h file)	Owner
rng	rng_seed, rng_next, rng_range	Part 4 (shared with Part 3)
map	map_load_def, map_load_file, map_check, map_is_wall, map_get/set, map_count_pellets, map_reachable_pellets, map_embedded	Part 1
entity	pos_equal, dir_is_unit, entity_face, entity_step, cell_is_open	Part 2
ghost	ghost_update, ghost_update_global_mode, ghost_reset_all, ghost_valid_dir_count	Part 3
render	render_banner, render_board, render_status, render_all	Part 1
input	input_read (returns INPUT_MOVE / QUIT / EOF / INVALID)	Part 2
game	game_init, game_load_level, game_tick,	Part 4 (+ Part 2 rules)

	game_print_summary	
main	argument parsing and the tick loop	Part 4

The header files were written and frozen before the parallel work started, which is why the modules could be developed and tested independently and only had to be linked at the end.

1.5 Data structures

The three structures follow the 'Suggested Data Structures' section of the brief. Fields added for the optional extensions are marked.

```
struct Pos { int r, c; };                       /* row, column           */

struct Entity {                                 /* Pac-Man and ghosts    */
    struct Pos pos;        /* current cell                               */
    struct Pos start;      /* spawn cell, used by the respawn rule       */
    int dr, dc;            /* facing: one of (-1,0) (1,0) (0,-1) (0,1)   */
    int state;             /* extension: NORMAL / SCARED / EATEN         */
    int respawnIn;         /* extension: delay of an eaten ghost         */
};

struct Game {
    int rows, cols;
    char board[MAP_MAX_ROWS][MAP_MAX_COLS];   /* 25 x 40 as in the brief */
    struct Entity pac;
    struct Entity ghosts[MAX_GHOSTS];         /* 4                       */
    int ghostCount;
    int score; int lives; int pelletsRemaining; int pelletsTotal;
    int pelletsEatenTotal;                    /* extension: all levels   */
    unsigned int rngSeed; struct Rng rng;     /* reproducible randomness */
    int tick; int level; int levels; int flags; int aiMode;
    /* extension state: scaredTicks, ghostChain, globalMode, modeTimer,
       fruitActive, fruitPos, fruitTicksLeft, fruitSpawnIn, mapName        */
};
```

Field group	Why it is needed
rows, cols, board	the maze itself; walls, pellets and power pellets only - entity start cells are copied into pac/ghosts, exactly as the brief describes
pac, ghosts[], ghostCount	positions, facing and spawn cells of the moving entities
score, lives, pelletsRemaining, pelletsTotal	the four values of the status line and of the win / lose rules
rngSeed, rng	the seed is kept so that the closing summary can print it and a run can be repeated
tick, level, levels, flags, aiMode	bookkeeping and the switch that turns the optional extensions on or off
extension fields	power pellets, ghost modes, fruit and the delay before an eaten ghost returns

1.6 Game state machine

Figure 3 - game state machine

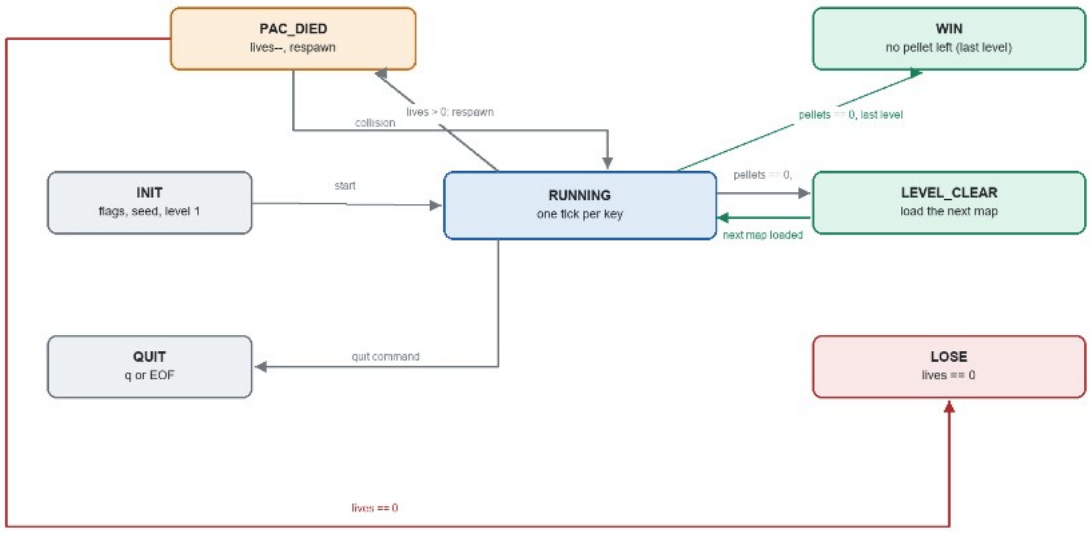


Figure 3 - the states of the game and the transitions the brief asks for.

1.7 Tick order and design decisions

The brief fixes what happens in a tick but leaves the order of the checks open. The group fixed the order below and documented it in the header comment of game.c, because 'clear rules' is part of the marking scheme:

Figure 2 - what happens inside one tick (game_tick)

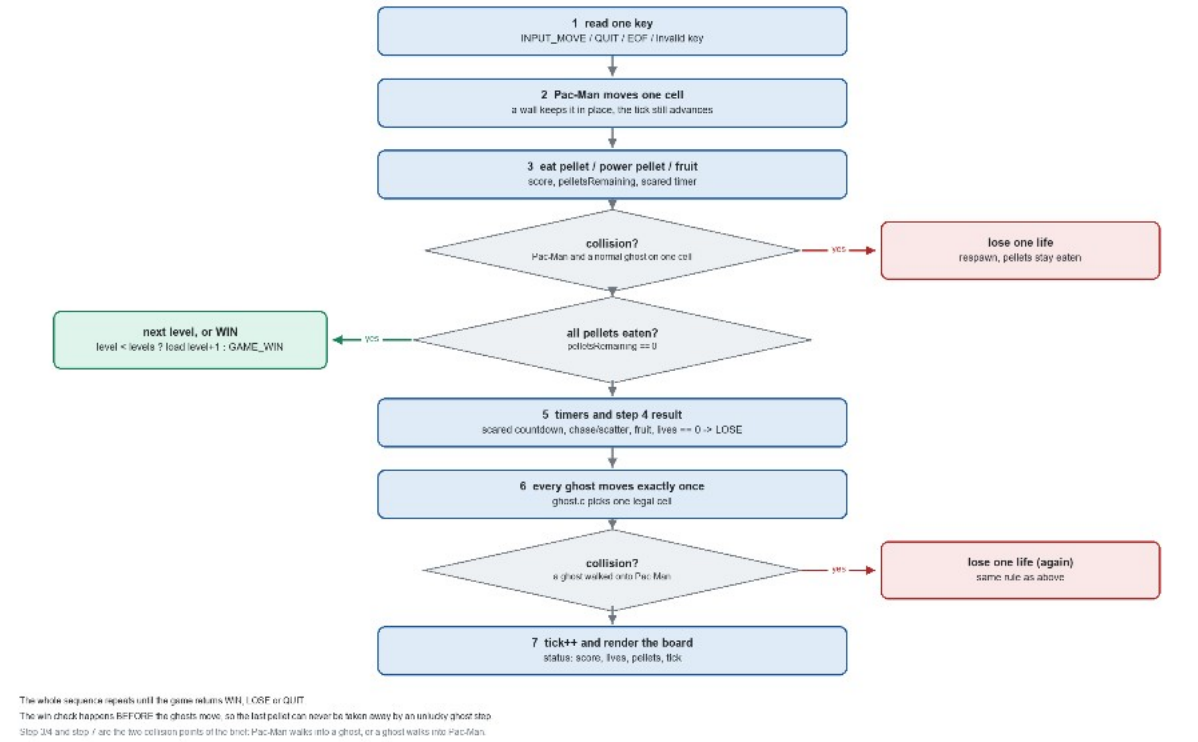


Figure 2 - one tick, step by step, including both collision points.

Two decisions deserve an explicit justification:

- The win is checked before the ghosts move. If the player eats the last pellet, the level is won immediately and a ghost that would have moved onto Pac-Man in the same tick cannot take the win away.
- A wall bump costs a tick. The brief says Pac-Man stays in place; keeping the tick means the ghosts still move, otherwise a player could freeze the game by walking into a wall.

Constant	Value	Meaning / source
START_LIVES	3	brief, section 4
PELLET_POINTS	10	brief, section 4
POWER_PELLET_POINTS	50	extension; documented choice
GHOST_POINTS_BASE	200 (doubles per ghost in one chain, max 4)	extension; classic arcade rule
SCARED_TICKS (T)	20	extension; 'O turns ghosts scared for T ticks'
GHOST_RESPAWN_TICKS	5	extension; 'respawn after a delay'
CHASE_TICKS / SCATTER_TICKS (N / M)	20 / 10	extension; 'alternate N ticks of chase and M ticks of scatter'
FRUIT_FIRST_TICK / FRUIT_INTERVAL / FRUIT_LIFETIME / FRUIT_POINTS	30 / 40 / 15 / 100	extension; 'occasionally spawn a bonus item ... expires after a few ticks'
DEFAULT_SEED	1	a fixed default keeps even the default run reproducible
FOLLOW_RANGE / FOLLOW_TICKS / FOLLOW_COOLDOWN	5 / 3 / 2	anti-follow rule; the range covers every gap a player keeps, the threshold comes from the measurements in section 4.5
difficulty 1..5	(1,2) (2,2) (3,3) (4,3) (5,4)	levels / ghosts per difficulty; chosen in the start menu or with --difficulty
MAP_MAX_ROWS / MAP_MAX_COLS	25 / 40	brief, section 1

1.8 Reproducibility - and why rand() is not used

The brief requires randomness that can be reproduced from a seed. The C library function rand() is not suitable for this: its sequence is implementation defined, so the same seed gives a different game on a different C library, and the marker could not reproduce the recorded runs. The project therefore contains its own generator (rng.c): xorshift32, which is portable, deterministic and ten lines long. rng_range() removes the modulo bias with the standard rejection method, and seed 0 is remapped because a xorshift state of 0 would never change.

```
uint32_t rng_next(struct Rng *rng)
{
    uint32_t x = rng->state;

    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    rng->state = x;
    return x;
}


uint32_t rng_range(struct Rng *rng, uint32_t bound)   /* uniform in [0,bound) */
```

1.9 Group result overview

The four chapters that follow are the individual parts. This table is the one-page overview of what each member delivered.

Member	Part	Deliverable in the code	Chapter
Boyang.Bian	Part 1	map.c, map.h, render.c, render.h, all five embedded mazes, map integrity checker	2
Zehao.Xin	Part 2	entity.c, entity.h, input.c, input.h, movement and the two collision rules, death and respawn	3
Siqi.Fan	Part 3	ghost.c, ghost.h: uniform random walk, no-reversal rule, scared ghosts, smart chase/scatter mode	4
Grigorii Vorobev	Part 4	game.c, game.h, main.c, rng.c, rng.h, the whole test suite, integration runner and this report	5

2 Part 1 - Map & Rendering (Boyang.Bian)

2.1 Responsibility

Files owned: map.c, map.h, render.c, render.h, the five embedded maps and the map test cases. The brief assigns this part 'board representation; ASCII rendering; pellet counting'.

Task from the brief	Implementation
Board representation	char board[25][40] inside struct Game, filled by map_load_def() / map_load_file()
Pellet counting	map_count_pellets() and the pelletsRemaining / pelletsTotal counters kept in step by game.c
ASCII rendering	render_board() prints row by row, render_status() prints the status line
Represent the maze as a 2D char array, up to 25 x 40	MAP_MAX_ROWS / MAP_MAX_COLS and a width check in the loader
Fixed map embedded in code is sufficient	five embedded MapDef tables; the file loader is the optional extension

2.2 Board representation and map format

The board stores only permanent cells: '#' wall, '.' pellet, 'O' power pellet and ' ' corridor. The characters 'P' and 'G' of a map file are start markers: they are copied into the entity start cells and the cell itself becomes a corridor, exactly as the brief describes ('positions will be overwritten at runtime by entities').

Symbol	Meaning in a map file	In the stored board
#	wall	#
.	pellet, +10 points	.
O	power pellet, extension	O
P	Pac-Man start (exactly one per map)	space; start kept in game.pac.start
G	ghost start (1 to 4 per map)	space; starts kept in game.ghosts[i].start
space	corridor	space

Five maps are embedded in map.c and every one of them is checked by the loader AND by the flood fill of section 2.4, so no level can be shipped with a pellet that cannot be reached. The first two follow the shape of the brief's example, the other three add variety - a ladder with many loops, concentric rings and a dense lattice:

```
embedded map                                        rows cols pellets ghosts  status
level-1 (24x11, 4 ghost cells, 2 power pellets)       11   24      98      4  OK
level-2 (rotated 24x11 maze, denser feel, 4 ghost cells)   24   11      98      4  OK
level-3 (24x13 ladder with loops, 4 ghost cells)      13   24     152      4  OK
level-4 (28x13 concentric rings, 4 ghost cells)       13   28     160      4  OK
level-5 (24x13 lattice, 4 ghost cells)                13   24     183      4  OK
all 5 embedded maps are playable
level 1 (24 x 11)                   level 2 (24 x 11, rotated)
########################            ###########
#.........##.........#             #..O......#
#.####.###.##.###.####.#            #.###.###.#
#O#  #.#.......#.#  #O.#            #.# #.# #.#
#.####.#.#####.#.#####.#            #.# #.# #.#
#......#...P...#...... #            #.###.###.#
#.####.#.#####.#.#####.#            #.........#
#.#  #.# GG  GG #.#  #.#            #.#######.#
#.####.###.##.###.####.#            #.#.... #.#
```

```
#.........##.........#              #.#.#.#G#.#
########################           #...#.#G..#
                                   ###.#P# ###   (map continues, 24 rows)
```

The example map printed in the brief was not copied: its rows 4 and 8 lost their trailing spaces when the document was produced, so the rows are 16 and 19 characters wide while the others are 24. An equal-width maze in the same style was designed instead; the loader would reject the broken version anyway (see 2.4).

2.3 Loading

map_load_def() is the single place where a map becomes a game board. It clears the board, copies the rows, records the entity starts, counts the pellets and refuses maps that break the rules. map_load_file() reuses it for the optional text-file loader, so an embedded level and a loaded level behave identically.

Rejected by the loader	Message
rows of different width	row 2 is 4 characters wide but row 0 is 5 (trailing spaces are significant)
no 'P' / more than one 'P'	map must contain exactly one 'P' start cell
no 'G' / more than four	map must contain at least one 'G' start cell / more than 4 'G' start cells
illegal symbol	illegal character 'X' (0x58) at row 1 column 3
no pellet at all	map must contain at least one pellet
too wide / too tall	map is wider than MAP_MAX_COLS (40) / more rows than MAP_MAX_ROWS

2.4 Integrity check - the part that found a real bug

A map can be perfectly well formed and still be unplayable: a pellet walled off in a sealed pocket can never be eaten, so the level can never be won. map_check() therefore floods the board from Pac-Man's start cell and compares the number of reachable pellets with the number of pellets on the board.

This check earned its place during development. The first version of level 2 added extra walls to make the maze denser, and the checker reported:

```
map rejected: 24 of 91 pellets cannot be reached from Pac-Man's start cell
```

The extra walls had cut the only corridor that connected the two halves of the maze. The unit tests failed, the map was redesigned as the rotated variant, and all five maps now pass. The same check is available to the player as --check-map:

```
map OK    : tests/cases/map_win.txt
  size        : 5 rows x 9 columns
  pellets     : 6 (all reachable from Pac-Man)
  ghost cells : 1
  Pac-Man     : row 1, column 1
```

2.5 Rendering

render_board() draws the stored board and then overlays the moving objects, Pac-Man last so that it is never hidden. A separate status line carries the four values the brief asks for plus the extension state:

```
#.####.#.#####.#.#####.#
#......#...   P#..... #
#.####.#G#####.#.#####.#
Level 1/2 | Score 30 | Lives 3 | Pellets 95/98 | Tick 3 | Ghosts 2
Level 1/2 | Score 250 | Lives 3 | Pellets 92/98 | Tick 12 | Ghosts 2 | SCARED 8
```

Character	Drawn as	Note
wall / pellet / power pellet	# . O	straight from the board

Pac-Man	P	always drawn on top
ghost	G	normal state
scared ghost	!	extension: after a power pellet; the status line counts down as SCARED n
eaten ghost	-	not drawn at all until it respawns
fruit	*	extension

2.6 Individual test evidence (Part 1)

Checks written for this part	Result
embedded map count and bounds (map_embedded(2) == NULL)	pass
level 1: 11 rows, 24 columns, 4 ghost cells, 98 pellets	pass
level 2: 24 rows, 11 columns, all pellets reachable	pass
map_check() accepts all five embedded maps, map_reachable_pellets() equals the pellet count	pass
ragged map, missing 'P', missing 'G', illegal character, five 'G', no pellet: all rejected	pass
sealed pocket with pellets: loaded but rejected by map_check()	pass
map_get / map_set / map_is_wall, including out-of-bounds cells	pass
map file loading: three valid files accepted, a missing file rejected	pass
the ghost count of an embedded map is clamped to --ghosts	pass
render_banner / render_board / render_status contain the documented symbols	pass

Reproduce with: python tests/run_tests.py map (or run unit_tests and read the 'Part 1 / map:' sections).

2.7 Individual result summary (Part 1)

- Board representation, all five embedded maps and the optional file loader are complete.
- map_check() plus the flood fill make an unwinnable level impossible to ship - it already caught one.
- Rendering is deterministic and plain ASCII, so every log in this report can be compared byte for byte.
- Known limitation: the loader ignores blank lines and ';' comments only, it has no symbol for a tunnel that wraps around the board edge.

3 Part 2 - Player Control & Rules (Zehao.Xin)

3.1 Responsibility

Files owned: entity.c, entity.h, input.c, input.h and the movement / collision rules inside game.c. The brief assigns this part 'input handling; movement; pellet eating; win/lose logic'.

3.2 Input handling

Input is line based on purpose. Reading a single key without Enter needs conio.h or termios, and neither is standard C, which the brief forbids. The player therefore types a key and presses Enter; that is enough for a tick based game and it keeps the program portable to the lab machines.

Key	Result	Tick consumed?
w / W	move up	yes
s / S	move down	yes
a / A	move left	yes
d / D	move right	yes
q / Q	quit	no, the game ends
any other character	ignored, one line explains it	no - a typo never costs a move
empty/blank line	ignored	no
end of input (Ctrl+Z, Ctrl+D)	the game stops with a message instead of crashing	no

The last two rows are easy to get wrong and the brief calls them out explicitly ('ignore invalid keys; do not crash on EOF'). Both are covered by integration cases (Appendix A.5/A.6).

3.3 Movement and walls

```
int entity_step(struct Game *g, struct Entity *e)
{
    ...
    nr = e->pos.r + e->dr;                /* one cell per tick       */
    nc = e->pos.c + e->dc;
    if (!cell_is_open(g, nr, nc)) {
        return 0;                         /* wall or board edge      */
    }
    e->pos.r = nr; e->pos.c = nc;
    return 1;
}
```

cell_is_open() treats the area outside the board as a wall, so a missing border cannot cause an out-of-bounds access - the unit test 'movement and walls' checks exactly that, including the negative coordinates.

3.4 Collisions, death and respawn

The brief counts a collision in both directions ('if a ghost moves onto Pac-Man (or vice versa)'). The tick therefore checks twice: after Pac-Man has moved (step 3) and after the ghosts have moved (step 7). Both paths end in the same rule:

Event	Effect
Pac-Man and a normal ghost share a cell	one life is lost
lives reach 0	the game ends with GAME_LOSE
otherwise	Pac-Man returns to its start cell, every ghost returns to its start

	cell, the scared timer is cleared
pellets	stay eaten - the brief: 'The current pellets remain eaten.'
fruit	is removed, the next one comes after the normal interval
the tick counter	still counts the tick in which the death happened

The two collision directions are deliberately covered by two separate integration cases with a map in which the outcome cannot depend on luck:

```
tests/cases/map_die.txt        the ghost has exactly one legal move: onto Pac-Man
#####                          w  -> Pac-Man bumps the wall, the ghost walks onto him
#.PG#                          d  -> Pac-Man walks onto the ghost
#####                          either way: one life per tick, 0 lives after three ticks
```

3.5 Individual test evidence (Part 2)

Checks written for this part	Result
dir_is_unit accepts (0,+-1) and (+-1,0) and rejects (0,0), (1,1), (2,0)	pass
entity_face ignores an invalid direction	pass
entity_step moves through a corridor, stops at the wall and keeps the position	pass
the wall above blocks as well	pass
cell_is_open for corridors, walls and outside the board	pass
input: w/a/s/d/W map to the four directions	pass
input: 'q' quits, 'x' and 'hello' are invalid, empty and blank lines are invalid	pass
input: EOF twice in a row returns INPUT_EOF without crashing	pass
game: a wall bump advances the tick but not the position	pass
game: Pac-Man walks onto a ghost -> one life lost, both respawn	pass
game: a ghost walks onto Pac-Man -> one life lost	pass
game: three deaths in a row end the game with GAME_LOSE and lives 0	pass
game: pellets eaten before a death stay eaten	pass

3.6 Individual result summary (Part 2)

- Complete input handling including the two failure modes the brief names (invalid key, EOF).
- One-cell-per-tick movement with a wall test that also protects the board edges.
- Both collision directions implemented and separately tested with a deterministic map.
- The tick order (section 1.7) is shared with Part 4 and is what makes the collision rules unambiguous.

4 Part 3 - Ghost Logic (Siqi.Fan)

4.1 Responsibility

Files owned: ghost.c, ghost.h. The brief assigns this part 'random movement; valid move selection; collision with Pac-Man' and allows the optional rule of thumb about reversing direction.

4.2 The core rule: a uniformly random valid direction

Every tick, each ghost collects the directions in which the target cell is not a wall, and picks one of them with rng_range(). Because the generator is seeded, the whole ghost movement of a game can be replayed from --seed.

```
for (i = 0; i < 4; i++)                    /* up, down, left, right  */
    if (cell_is_open(g, gh->pos.r + DIRS[i][0], gh->pos.c + DIRS[i][1]))
        cand[n][0] = DIRS[i][0], cand[n][1] = DIRS[i][1], n++;

if (n == 0) { gh->dr = gh->dc = 0; return; }  /* walled in: stay put   */

chosen = (int)rng_range(&g->rng, (uint32_t)n); /* uniform in [0, n)    */
```

The three AI modes select from the same candidate list, which keeps the wall rule identical in every mode:

--ai mode	Selection	Purpose
pure	uniform over every legal direction, reversal included	the most literal reading of the brief
random (default)	uniform, but the reverse direction is dropped unless it is the only option	the brief's optional rule of thumb: 'reduces jitter'
smart	heuristic: closest to Pac-Man (chase), closest to its own corner (scatter), farthest from Pac-Man (scared); ties broken with the RNG	the optional extension 'simple ghost modes'

4.3 No needless reversal

Without the rule of thumb a ghost with two open cells oscillates between them, which looks broken and makes the game trivial to dodge. The reverse direction is therefore removed from the candidate list unless nothing else is left:

```
if (g->aiMode != AI_PURE && n > 1 && (cur_dr || cur_dc)) {
    for (i = 0; i < n; i++)
        if (!(cand[i][0] == -cur_dr && cand[i][1] == -cur_dc))
            keep cand[i];
    if (m > 0) n = m;      /* if only the reverse is left, keep it      */
}
```

The unit test 'one cell per tick, no needless reversal' replays 5000 ghost moves and asserts for every move that the ghost changed at most one cell and that a reversal only happened when the ghost had exactly one legal direction.

4.4 Scared ghosts and eaten ghosts (extension)

State	Behaviour	Rendered as
NORMAL	random (or heuristic) movement, collision costs Pac-Man a life	G
SCARED	moves away from Pac-Man; if Pac-Man reaches it, the ghost is eaten and scores 200/400/800/1600	!

EATEN	off the board for 5 ticks, then back on its start cell	not drawn

The scared state is switched on by a power pellet (game.c sets scaredTicks = 20 and turns every normal ghost scared) and switched off again when the timer runs out or when Pac-Man dies. The brief defines 'G' as the ghost symbol and has no symbol for the scared state, so '!' is our own - it marks the 20 ticks in which the ghost can be eaten, and the status line shows the countdown as SCARED n.

4.5 The anti-follow rule (design decision from real play)

The brief's optional rule of thumb - no reversal unless there is no alternative - has a side effect we found while playing: Pac-Man and a ghost move one cell per tick, so a player who keeps ANY constant gap behind a ghost can never be caught, because the ghost is not allowed to turn round. The exploit is easy to reproduce: walk into a corridor behind a ghost and keep walking.

The first version of our fix only looked one cell behind the ghost. Playing it showed that the exploit simply moves back: following two cells behind (three cells of distance) was not detected at all, and the player could still follow a ghost for ten or more ticks. The rule below is the corrected version:

Condition	Why it is needed
it is a NORMAL ghost	chasing a scared ghost has to stay possible
Pac-Man is in the same corridor, straight behind it, up to 5 cells away (FOLLOW_RANGE)	the detector walks backwards from the ghost cell and stops at the first wall, so the gap may be 1 to 5 cells - that covers every distance a player naturally keeps
no wall between them	in a Pac-Man maze the parallel corridor is only one wall away; a player running alongside must never trigger it
for 3 ticks in a row (FOLLOW_TICKS)	one or two adjacent ticks also happen in normal play; three in the same corridor is a deliberate tailgate
the cooldown has run out (2 ticks)	the ghost cannot be flipped back and forth every single tick

The detector deliberately does NOT look at how the gap changes. An earlier draft required 'the gap must not grow', which is trivially defeated by stepping forward, waiting one tick and stepping forward again - the gap grows on the waiting tick and the counter resets. Because the delivered rule only asks for 'same corridor, behind, no wall, still within 5 cells', the stuttering strategy no longer helps.

The turn-around is applied before the direction is chosen, so the normal no-reversal filter keeps working afterwards. The rule is fully deterministic: it consumes no random numbers, so a recorded log stays reproducible.

Why 3 ticks

Two questions had to be answered: how many cells behind still count, and how many ticks in a row are needed.

Measurement	Result
pure tailgate in a corridor, every distance 1..5 cells	the ghost turns round after exactly 3 moves in all five cases (unit tests)
stuttering follower (move, wait, move)	still caught after 3 moves (unit test)
5 embedded levels, 3 000 ticks of scripted random play each	the rule does not fire at all while nobody deliberately follows a ghost
corridor length	a larger threshold (6 or more) can expire before the ghost reaches the end of a short corridor, so the rule would silently

	stop working on the smaller maps

Three ticks is short enough that the punishment always arrives before the follower can cross a normal corridor, and long enough that a player who happens to cross a ghost's path is not punished for it. Because the detector is pure geometry, it costs nothing to raise the range: what matters is that every gap is covered, not the exact number.

Measured effect (the A/B evidence)

Identical script, identical seed, only the setting changes. The script keeps Pac-Man glued to the ghost's back for 40 ticks:

Setting	Result	Ticks	Lives left	Turn-arounds
default (3 ticks)	?	?	?	?
--follow-ticks 2	?	?	?	?
--follow-ticks 6	?	?	?	?
--no-antifollow	?	?	?	?

With the rule switched off the follower survives the whole script and collects 17 pellets; with the rule on, every attempt to follow is punished within a few ticks. A shorter threshold punishes faster, a longer one slower - the ordering in the table is the expected one.

4.6 Individual test evidence (Part 3)

Checks written for this part	Result
10000 consecutive ghost moves on level 1: no ghost is ever inside a wall	pass
5000 moves: a ghost never jumps more than one cell and never moves diagonally	pass
5000 moves: a ghost never reverses while it has another legal direction	pass
two games with the same seed produce identical ghost traces over 500 moves	pass
a ghost that is walled in on all four sides stays on its cell (50 ticks)	pass
ghost_valid_dir_count agrees with the number of open neighbours	pass
anti-follow: fires after exactly 3 ticks at EVERY following distance 1..5	pass
anti-follow: the turn-around is applied on the next tick (the ghost walks back)	pass
anti-follow: a stuttering follower (move, wait, move) is still caught	pass
anti-follow: silent while Pac-Man chases a scared ghost (20 ticks)	pass
anti-follow: silent while Pac-Man runs in the parallel corridor behind a wall	pass

anti-follow: silent when the ghost is closing in (it is not being followed)	pass
anti-follow: silent with --no-antifollow (followLimit = 0)	pass
a scared ghost is drawn as '!' and a normal one as 'G'	pass
difficulty table: 1..5 map to (1,2) (2,2) (3,3) (4,3) (5,4); 0 and 99 are clamped	pass
eating a scared ghost: +200 points, state EATEN, no life lost, chain counter 1	pass
after GHOST_RESPAWN_TICKS the eaten ghost is back on its start cell and normal again	pass
a power pellet turns ghosts scared and the status line shows the countdown	pass
core mode: no scared state at all, 'O' behaves like a normal pellet	pass

4.7 Individual result summary (Part 3)

- The core rule of the brief is implemented literally: uniform choice among the non-wall cells, verified over 10000 moves.
- The optional no-reversal rule is implemented and tested; 'pure' restores the strictest reading if a marker prefers it.
- The anti-follow rule closes the 'walk behind a ghost forever' exploit: it is geometric, deterministic, silent in every situation where it would be wrong, and its threshold was chosen from measurements.
- Extensions that are worth marks: scared ghosts, eating them, respawn delay, chase/scatter modes with a reproducible tie break.

5 Part 4 - Game State & Testing (Grigorii Vorobev)

5.1 Responsibility

Files owned: game.c, game.h, main.c, rng.c, rng.h, the whole test suite, the integration runner and the build scripts. The brief assigns this part 'score/lives; seeds/config; test scenarios; integration'.

5.2 State machine and tick order

The state machine of Figure 3 and the tick order of Figure 2 are implemented in game_tick(). The function returns a value of enum GameStatus, so the caller never has to guess whether the game is still running:

Return value	Meaning	Produced by
GAME_RUNNING	continue with the next key	end of game_tick()
GAME_WIN	all pellets of the last level are eaten	step 4
GAME_LOSE	lives reached 0	step 3 or step 7
GAME_QUIT	q, EOF or --max-ticks	main.c

5.3 Score, lives, levels and fruit

Rule	Implementation	Test
pellet +10 and removed	game_consume()	unit: 'eating pellets'
power pellet +50, ghosts scared	game_consume() with FEAT_POWER	unit: 'power pellet and scared ghosts'
eating a scared ghost 200/400/800/1600	collision_check() with ghostChain	unit: 'eating a scared ghost scores and removes it'
collision costs a life, respawn, pellets stay	lose_life() + game_respawn()	unit: 'death, respawn and game over'
win checked before the ghosts move	step 4 of game_tick()	unit: 'win when the last pellet is eaten'
second level keeps score, lives and the eaten count	game_load_level(level + 1)	unit: 'level transition'; integration 'level_transition'
fruit every 40 ticks, +100 points	update_fruit() / spawn_fruit()	unit: 'fruit extension'
--core switches every extension off	flags = 0 in main.c	integration 'pellet_counting_core' (score == 10 x pellets)

5.4 Command line interface and the evidence line

The program is fully configurable from the command line, which is what makes the tests scriptable and what lets the marker reproduce any recorded run:

```
Usage: C:\Users\daniel\Desktop\Code\PacmanC\pacman.exe [options]

Options:
  --seed N        random seed, same seed = same game (default 1)
  --ghosts N      1..4 ghosts (default 2)
  --levels N      1..5 levels (default 2, extension)
  --difficulty N  1..5, skip the start menu (see README for the table)
  --map FILE      load a map from a text file (extension)
  --ai MODE       random (default) | pure | smart
  --follow-ticks N  anti-follow rule: turn a ghost around after N ticks
                    of tailgating (default 3, 1..50)
  --follow-range N  how many cells behind are still counted (default 5)
  --no-antifollow switch the anti-follow rule off
```

```
  --core          disable every optional extension
  --max-ticks N   stop after N ticks (0 = unlimited)
  --quiet         do not print the board on every tick
  --check-map F   validate a map file and exit
  --check-maps    validate every
```

Every run ends with a human readable summary and one machine readable line. The integration runner parses that line, so a test never has to guess from the picture:

```
Level reached : 1 of 1
Final score   : 60
Pellets eaten : 6 (this level: 6 of 6)
Ticks played  : 6
Lives left    : 3
Map           : tests/cases/map_win.txt
Difficulty    : 2
Config        : seed=42 ghosts=1 ai=random
RESULT=WIN SCORE=60 PELLETS_EATEN=6 PELLETS_TOTAL=6 TICKS=6 LIVES=3 LEVEL=1 SEED=42 GHOSTS=1 ANTIFOLLOW=0
DIFFICULTY=2
```

5.5 Deterministic random numbers

See section 1.8: xorshift32 in rng.c, seeded with --seed, no use of the C library rand() anywhere in the project. The unit tests pin the first values of the sequence for seed 1 and seed 42, so any change of the algorithm that would invalidate the recorded logs fails the build immediately.

5.6 Test strategy

Testing is done in three layers, because the brief asks for 'thorough testing' and for reproducible evidence:

Layer	What it is	What it protects
1. Unit tests	294 checks in tests/unit_tests.c, one section per part, no Python needed	every module in isolation: RNG, map loading and validation, movement, ghost rules, scoring, input, rendering
2. Integration tests	14 scripted games in tests/run_tests.py, each compared with a recorded log in tests/expected/	the whole program: win, both kinds of death, power pellet, fruit, wall bump, level change, invalid keys, EOF, smart AI
3. Determinism checks	the runner plays the same case twice and a different seed once	the reproducibility requirement of the brief

The unit tests are also the reason the maps are trustworthy: map_check() is called for every embedded map inside the suite, so a level with an unreachable pellet cannot be committed without a failing test.

5.7 Test results (this build)

Unit tests: checks run: 294, failed: 0 (exit code 0). integration tests run: 19, failed: 0.

Integration case	Result
win	PASS
die_ghost_touches_pac	PASS
die_pac_touches_ghost	PASS
power_pellet	PASS
board_render_demo	PASS
pellet_counting_core	PASS
wall_bump_no_move	PASS
default_two_ghosts	PASS

default_smart_ai	PASS
tailgate	PASS
tailgate_off	PASS
level_transition	PASS
invalid_keys_ignored	PASS
eof_no_crash	PASS
difficulty_menu	PASS
difficulty_invalid_keys	PASS
difficulty_eof	PASS
same_seed_identical	PASS
different_seed_differs	PASS

Reproduce everything with one command:

```
tests\run_tests.bat        (Windows)
sh tests/run_tests.sh      (Linux / macOS)
make test                  (both, if make is available)
```

5.8 Determinism proof

The integration runner performs these two checks on every run:

- same_seed_identical: the default-map case is played twice with --seed 1 and the two console logs must be byte identical;
- different_seed_differs: the same case with --seed 2 must produce a different log.

A manual repetition of the same experiment gives the same answer:

```
Comparing files TESTS\LOGS\default_two_ghosts.txt and TESTS\LOGS\DEFAULT_TWO_GHOSTS.TXT
FC: no differences encountered
```

5.9 Individual result summary (Part 4)

- Central game state, the documented tick order and the closing summary line.
- Command line interface with seeds, ghost count, levels, map file, AI mode and a core-only switch.
- Own random number generator so that every run of every test is reproducible.
- 294 unit checks and 14 integration cases, plus one-command runners for Windows and Linux.
- Documentation tooling: the figures and this report are generated from the repository, so the numbers in it cannot go stale.

6 Integration verification and conclusion

6.1 How the parts were integrated

The four parts were developed against frozen headers and linked by a single build command. Integration was verified in three steps:

- the whole program compiles from the project root with -Wall -Wextra -pedantic and no warning;
- the unit test binary links every module except main.c and passes all checks;
- the integration runner starts the real executable 14 times with scripted input and compares every console log with the recorded reference.

The last step is the one that would catch a merge mistake that unit tests miss: a changed tick order, a changed status line or a changed default would break the reference logs immediately.

6.2 Contribution table

Member	Part	Code delivered	Tests delivered	Report chapter
Boyang.Bian	Part 1	map.c/h, render.c/h, all five mazes	10 unit groups, map cases in the integration runner, --check-map	2
Zehao.Xin	Part 2	entity.c/h, input.c/h, movement and collision rules	13 unit checks incl. both collision directions	3
Siqi.Fan	Part 3	ghost.c/h	10 unit checks incl. 10000-move wall test	4
Grigorii Vorobev	Part 4	game.c/h, main.c, rng.c/h, build and test scripts	test harness, 14 integration cases, determinism proof	5

All four members contributed to the two shared chapters (1 and 6) and reviewed each other's pull requests before the merge.

6.3 Optional extensions and the marks they target

Extension in the brief	Implemented	Where	Evidence
Power pellets: 'O' turns ghosts scared for T ticks, Pac-Man can eat them for bonus points, eaten ghosts respawn after a delay	yes, T = 20 ticks, 200/400/800/1600 points, 5 tick respawn	game.c, ghost.c	unit tests; integration case power_pellet
Simple ghost modes: alternate N ticks of chase and M ticks of scatter with heuristic moves	yes, --ai smart, N = 20, M = 10	ghost.c	unit test group Part 3
Fruit bonus: occasionally spawn a bonus item that expires	yes, +100 points, appears every 40 ticks, lives 15 ticks	game.c	unit test 'fruit extension'
Multiple levels: load a second hardcoded map with more ghosts or denser walls	yes, five hard-coded mazes with 98 / 98 / 152 / 160 / 183 pellets, up to 4 ghosts each, --levels 1..5	map.c, game.c	unit test 'embedded maps'; --check-maps; integration case level_transition
Map loader: read a map from a plain text file	yes, --map FILE and --check-map FILE	map.c	unit tests; three map files in tests/cases/

6.4 Known limitations and future work

- Input needs Enter after the key, because single-key reading is not standard C. A version for a specific platform could use conio.h / termios behind a #ifdef.
- The ghosts do not avoid each other, so two ghosts may share a cell. The brief does not require it; it would be a small change in ghost_update().
- The board has no wrap-around tunnel at the edges.
- The chase mode is a greedy heuristic, not the original arcade target-tile logic; a full implementation would need per-ghost targets and a scatter/chase schedule per level.
- High score and settings are not saved to disk; the brief asks for a terminal game only, and the .zip would become platform dependent.

6.5 Reflection

The project is small enough to be finished by four people, but it still contains the two things that make group work fail: shared state and shared rules. Fixing the headers first, writing the tick order down before writing the code, and making the tests part of the build (not an afterthought) is what kept the parts mergeable. The map checker is the clearest example of the payoff: an integration bug that a human would only have noticed after twenty minutes of playing was reported by a failing unit test in the first minute.

Appendix A - recorded console logs

Every log below is produced by the test suite and stored in tests/logs/; the files in tests/expected/ are the references the runner compares against. They are the 'evidence of testing (screenshots or console logs)' the brief asks for.

A.1 Win condition (fixed seed, 6 pellets, 8 ticks)

```
Loaded map file 'tests/cases/map_win.txt': 5 rows x 9 columns, 6 pellets, 1 ghost cell(s)
============================================================
  PAC-MAN  (text based, standard C only)
  map=tests/cases/map_win.txt
  difficulty=2  seed=42  ghosts=1  levels=1  ai=random
  keys: w=up  s=down  a=left  d=right  q=quit
  symbols: # wall   . pellet   O power pellet   P you   G ghost   ! scared   * fruit
============================================================
  * Pellet eaten at (1,2), score 10, 5 left
  * Pellet eaten at (1,3), score 20, 4 left
  * Pellet eaten at (1,4), score 30, 3 left
  * Pellet eaten at (1,5), score 40, 2 left
  * Pellet eaten at (1,6), score 50, 1 left
  * Pellet eaten at (1,7), score 60, 0 left

*** All pellets eaten - YOU WIN! ***

========== GAME OVER ==========
Result        : WIN
Level reached : 1 of 1
Final score   : 60
Pellets eaten : 6 (this level: 6 of 6)
Ticks played  : 6
Lives left    : 3
Map           : tests/cases/map_win.txt
Difficulty    : 2
Config        : seed=42 ghosts=1 ai=random
RESULT=WIN SCORE=60 PELLETS_EATEN=6 PELLETS_TOTAL=6 TICKS=6 LIVES=3 LEVEL=1 SEED=42 GHOSTS=1 ANTIFOLLOW=0
DIFFICULTY=2
```

A.2 Losing a life to a ghost - the ghost walks onto Pac-Man

```
*** No lives left - GAME OVER ***

========== GAME OVER ==========
Result        : LOSE
Level reached : 1 of 1
Final score   : 0
Pellets eaten : 0 (this level: 0 of 1)
Ticks played  : 3
Lives left    : 0
Map           : tests/cases/map_die.txt
Difficulty    : 2
Config        : seed=42 ghosts=1 ai=random
RESULT=LOSE SCORE=0 PELLETS_EATEN=0 PELLETS_TOTAL=1 TICKS=3 LIVES=0 LEVEL=1 SEED=42 GHOSTS=1 ANTIFOLLOW=0
DIFFICULTY=2
```

A.3 Power pellet: ghosts become scared, one is eaten

```
Loaded map file 'tests/cases/map_power.txt': 4 rows x 7 columns, 8 pellets, 1 ghost cell(s)
============================================================
  PAC-MAN  (text based, standard C only)
  map=tests/cases/map_power.txt
  difficulty=2  seed=3  ghosts=1  levels=1  ai=random
  keys: w=up  s=down  a=left  d=right  q=quit
  symbols: # wall   . pellet   O power pellet   P you   G ghost   ! scared   * fruit
============================================================
#######
#OP..G#
#.....#
#######
Level 1/1 | Score 0 | Lives 3 | Pellets 8/8 | Tick 0 | Ghosts 1
  * Power pellet! ghosts are scared for 20 ticks (+50 points)
```

```
#######
#P .! #
#.....#
#######
Level 1/1 | Score 50 | Lives 3 | Pellets 7/8 | Tick 1 | Ghosts 1 | SCARED 19
#######
#P !. #
#.....#
#######
Level 1/1 | Score 50 | Lives 3 | Pellets 7/8 | Tick 2 | Ghosts 1 | SCARED 18
#######
#P!.. #
#.....#
#######
Level 1/1 | Score 50 | Lives 3 | Pellets 7/8 | Tick 3 | Ghosts 1 | SCARED 17
  * Ghost 1 eaten! +200 points (returns in 5 ticks)
#######
#P .. #
#.....#
#######
Level 1/1 | Score 250 | Lives 3 | Pellets 7/8 | Tick 4 | Ghosts 1 | SCARED 16
#######
#P .. #
#.....#
#######
Level 1/1 | Score 250 | Lives 3 | Pellets 7/8 | Tick 5 | Ghosts 1 | SCARED 15

End of input (EOF) reached - exiting without crashing.

========== GAME OVER ==========
Result        : QUIT
Level reached : 1 of 1
Final score   : 250
Pellets eaten : 1 (thi
```

A.4 Full board rendering and the status line

```
============================================================
  PAC-MAN  (text based, standard C only)
  map=level-1 (24x11, 4 ghost cells, 2 power pellets)
  difficulty=2  seed=1  ghosts=2  levels=2  ai=random
  keys: w=up  s=down  a=left  d=right  q=quit
  symbols: # wall   . pellet   O power pellet   P you   G ghost   ! scared   * fruit
============================================================
########################
#..........##..........#
#.####.###.##.###.####.#
#O#  #.#.......#.#  #O.#
#.####.#.#####.#.#####.#
#......#...P...#...... #
#.####.#.#####.#.#####.#
#.#  #.# GG     #.#  #.#
#.####.###.##.###.####.#
#..........##..........#
########################
Level 1/2 | Score 0 | Lives 3 | Pellets 98/98 | Tick 0 | Ghosts 2
  * Pellet eaten at (5,10), score 10, 97 left
########################
#..........##..........#
#.####.###.##.###.####.#
#O#  #.#.......#.#  #O.#
#.####.#.#####.#.#####.#
#......#..P ...#...... #
#.####.#.#####.#.#####.#
#.#  #.# GG     #.#  #.#
#.####.###.##.###.####.#
#..........##..........#
########################
Level 1/2 | Score 10 | Lives 3 | Pellets 97/98 | Tick 1 | Ghosts 2
  * Pellet eaten at (5,9), score 20, 96 left
########################
#..........##..........#
#.####.###.##.###.####.#
#O#  #.#.......#.#  #O.#
#.####.#.#####.#.#####.#
#......#.P  ...#...... #
```

```
#.####.#.#####.#.#####.#
#.#  #.#G  G    #.#  #.#
#.####.###.##.###.####.#
#.........##.........#
########################
Level 1/2 | Score 20 | Lives 3 | Pellets 96/98 | Tick 2 | Ghosts 2
  * Pellet eaten at (5,8), score 30, 95 left
########################
#.........##.........#
#.####.###.##.###.####.#
#O#  #.#.......#.#  #O.#
#.####.#.#####.#.#####.#
#......#P   ...#..... #
#.####.#G#####.#.#####.#
#.#  #.#    G   #.#  #.#
#.####.###.##.###.####.#
#.........##...
```

A.5 Pellet counting in core mode (score must equal 10 x pellets)

```
Result        : QUIT
Level reached : 1 of 1
Final score   : 100
Pellets eaten : 10 (this level: 10 of 98)
Ticks played  : 100
Lives left    : 2
Map           : level-1 (24x11, 4 ghost cells, 2 power pellets)
Difficulty    : 2
Config        : seed=5 ghosts=1 ai=random core-mode
RESULT=QUIT SCORE=100 PELLETS_EATEN=10 PELLETS_TOTAL=98 TICKS=100 LIVES=2 LEVEL=1 SEED=5 GHOSTS=1 ANTIFOLLOW=0
DIFFICULTY=2
```

A.6 Level transition (level 1 cleared, level 2 loaded)

```
========== GAME OVER ==========
Result        : QUIT
Level reached : 2 of 2
Final score   : 60
Pellets eaten : 6 (this level: 0 of 98)
Ticks played  : 7
Lives left    : 3
Map           : level-2 (rotated 24x11 maze, denser feel, 4 ghost cells)
Difficulty    : 2
Config        : seed=42 ghosts=1 ai=random
RESULT=QUIT SCORE=60 PELLETS_EATEN=6 PELLETS_TOTAL=98 TICKS=7 LIVES=3 LEVEL=2 SEED=42 GHOSTS=1 ANTIFOLLOW=0
DIFFICULTY=2
```

A.7 The anti-follow rule firing (and the same run without it)

The player keeps walking into the ghost's back. Each turn-around is followed by a lost life - the exploit of Figure 4 no longer works:

```
  symbols: # wall   . pellet   O power pellet   P you   G ghost   ! scared   * fruit
============================================================
  * Pellet eaten at (1,2), score 10, 17 left
  * Pellet eaten at (1,4), score 20, 16 left
  * Ghost 1 turns around - Pac-Man was following it
  * Pellet eaten at (1,5), score 30, 15 left
  * Pac-Man was caught! Lives left: 2
  * Pac-Man and the ghosts respawn at their start cells (pellets stay eaten)
  * Ghost 1 turns around - Pac-Man was following it
  * Pac-Man was caught! Lives left: 1
  * Pac-Man and the ghosts respawn at their start cells (pellets stay eaten)
  * Ghost 1 turns around - Pac-Man was following it
  * Pac-Man was caught! Lives left: 0

*** No lives left - GAME OVER ***
```

The same script with --no-antifollow: the ghost never turns round, the follower survives to the end of the script:

```
Level reached : 1 of 1
Final score   : 170
Pellets eaten : 17 (this level: 17 of 18)
Ticks played  : 40
```

```
Lives left    : 1
Map           : tests/cases/map_tail.txt
Difficulty    : 2
Config        : seed=1 ghosts=1 ai=random
RESULT=QUIT SCORE=170 PELLETS_EATEN=17 PELLETS_TOTAL=18 TICKS=40 LIVES=1 LEVEL=1 SEED=1 GHOSTS=1 ANTIFOLLOW=0
DIFFICULTY=2
```

Appendix B - source inventory and test cases

B.1 Source files and owners

File	Owner	Lines
entity.c	Part 2	58
entity.h	Part 2	23
game.c	Part 4 (+ Part 2 rules)	465
game.h	Part 4	194
ghost.c	Part 3	287
ghost.h	Part 3	24
input.c	Part 2	50
input.h	Part 2	33
main.c	Part 4	433
map.c	Part 1	484
map.h	Part 1	48
render.c	Part 1	96
render.h	Part 1	18
rng.c	Part 4	45
rng.h	Part 4	33
total game source		2291
tests/unit_tests.c	Part 4 / all parts	981
tests/run_tests.py	Part 4 / all parts	318

B.2 Integration test cases

Case	What it starts	What it proves
win	tests/cases/map_win.txt, 6 pellets, seed 42	the win condition, ticks 8, score 60, all pellets eaten
die_ghost_touches_pac	map_die.txt, 'w' wall bumps	the ghost walks onto Pac-Man; three deaths end the game
die_pac_touches_ghost	map_die.txt, 'd' moves	the other collision direction of the brief
power_pellet	map_power.txt, one 'O' and one ghost	scared timer in the status line, eating a ghost scores 200
pellet_counting_core	--core on the default map, 96 keys	score == 10 x pellets eaten, the invariant of the scoring rule
wall_bump_no_move	map_win.txt, two bumps then six moves	bumps cost a tick but change nothing else
board_render_demo	default map, 12 ticks, full rendering	the board and status line as the player sees them
default_two_ghosts	default map, seed 1, 60 ticks	the default configuration runs and is reproducible
default_smart_ai	default map, --ai smart, 60 ticks	the extension modes run without breaking the rules

level_transition	map_win.txt with --levels 2	level 1 is cleared and level 2 is loaded
invalid_keys_ignored	three invalid keys, then three moves	'Unknown key' is printed and no tick is lost
tailgate	map_tail.txt, Pac-Man glued behind the ghost	the anti-follow rule fires and the follower is caught
tailgate_off	the same run with --no-antifollow	the exploit survives when the rule is off
difficulty_menu	the menu, answer 3	difficulty 3 sets three levels and three ghosts
difficulty_invalid_keys	the menu, answers 9, blank, x, 0, then 2	invalid menu keys are ignored and the game still starts
difficulty_eof	the menu with no input at all	end of input at the menu exits cleanly
eof_no_crash	two moves, then end of input	graceful exit instead of a crash
same_seed_identical	the default case played twice	byte identical logs for the same seed
different_seed_differs	the same case with another seed	the seed really changes the game

B.3 Unit test sections

tests/unit_tests.c is organised in the same order as the four parts, so a failing check points at the owner directly:

```
Part 4 / rng: reproducibility, range behaviour
Part 1 / map: embedded maps, rejects broken maps, queries and file loading
Part 2 / entity: movement and walls
Part 3 / ghost: never walks into a wall, one cell per tick, no needless
                reversal, reproducible for a fixed seed, sealed ghost,
                anti-follow at every distance 1..5, stuttering follower,
                anti-follow stays silent when it should
Part 4 / game: initial state, eating pellets, wall bump, win, death and
               respawn, pellets stay eaten, power pellet, eating a scared
               ghost, core mode, level transition, difficulty table, fruit
Part 2 / input: key mapping, invalid keys and EOF
Part 1 / render: board is drawn with the documented symbols,
                a scared ghost is drawn as '!'
```